Exercices de Compléments d'algorithmique

Série 1 – Structures de données et opérations sur celles-ci
(20 février 2026)

Département d'Informatique – Faculté des Sciences – UMONS

Pré-requis : Cours jusqu'au **Chapitre 2**.

Objectifs : Se familiariser avec la sélection de structures de données pour un problème donné; s'entraîner à manipuler une structure de données plus complexe; se familiariser avec les notions de mapping, filtrage, réduction ou compréhension de liste.

---

1 Exercices sur les structures de données

1.1 DS Academy

Avant de savoir les implémenter et les utiliser, le plus important avec les structures de données, c'est de savoir qu'elle existent et de quoi elles sont capables.

Plus vous en connaîtrez, plus il risque d'être difficile de faire un choix, il peut donc être très utile de s'entraîner à choisir quelle structure de données correspond à une situation donnée.

Pour vous entraîner, vous décidez de passer le test d'entrée de l'académie des structures de données (DS Academy en anglais). Vous avez l'impression de vous retrouver dans un film d'arts martiaux où la·le protagoniste doit réaliser un tas de tâches qui n'ont, au premier abord, rien à voir avec son entraînement. Ce n'est qu'à la fin de votre entraînement que vous finirez par comprendre pourquoi on vous envoie faire les courses ou remplir des carnets d'adresse : pour mieux pouvoir choisir vos structures de données. Justement, pour terminer votre entraînement, on vous demande de faire exactement cela !

1 Pour chacune des situations suivantes, choisissez une strucuture de données parmi les chaînes, les tuples, les listes, les dictionnaires, les ensembles ou les frozen sets. Expliquez à chaque fois brièvement votre raisonnement. Si une situation vous semble ambigüe, n'hésitez pas à préciser comment l'ambigüité affecte votre choix.

- Vous devez stocker un historique des actions d'un utilisateur sur un site web. Vous voulez être certain de pouvoir retrouver quel événement en précède un autre.
- Vous voulez stocker les constantes d'une application. Comme toute constante, celles-ci ne seront jamais modifiées. On souhaite uniquement conserver les valeurs des constantes, pas leur nom ou quoi que ce soit d'autre.
- Vous devez enregistrer les coordonnées (x, y) d'un point sur un graphique. A priori, ce point ne se déplacera pas sur le graphique, celles-ci ne changeront donc jamais.
- Vous devez stocker une liste d'options sélectionnées par un utilisateur sur une page web. L'utilisateur peut ajouter ou enlever des options.
- Vous devez créer un registre d'emprunt de livre où chaque livre est associé à la personne qui vous l'a emprunté.
- Vous devez stocker l'ADN d'un organisme sous forme d'une séquence de nucléotides (A, T, C, G).

Vous avez passé le premier examen mais la *DS Academy* veut maintenant savoir si vous faites partie des meilleurs.

2 Faites la même chose que précédemment, mais faites attention, maintenant vous devez utiliser des combinaisons de structures de données. Par exemple, vous pourriez vouloir utiliser une liste où chaque élément est en fait un ensemble. Pour chacune, écrivez un exemple de déclaration d'une telle structure de données en Python.

— Vous devez gérer une base de données de recettes de cuisine. Chaque recette est représentée par une liste fixe d'ingrédients, et vous voulez éviter d'avoir des doublons de recettes.
— Vous devez enregistrer les notes des étudiants pour plusieurs matières. Chaque étudiant étant identifié par son nom et son prénom. On aimerait avoir accès rapidement à la note d'un étudiant pour une matière donnée en connaissant son nom, son prénom, et le nom de la matière.
— Vous devez organiser des programmes télévisés par catégorie. Chaque catégorie (comme "sport", "documentaire", "divertissement") est associée à un ensemble d'émissions spécifiques. Vous voulez garantir que la structure contenant les catégories et leurs émissions ne soit pas modifiable après définition.

1.2 Capitaine Edward

Edward, votre connaissance de la *DS Academy*, est passionné par les pirates. Il souhaite créer une chasse au trésor et vous demande de l'aide pour l'organisation de celle-ci. Il souhaite réaliser des cartes qui notent la valeur des trésors enterrés aux différents endroits où aura lieu la chasse. Il aimerait que vous prépariez un programme qui permette de garder en mémoire ces cartes. Une carte est modélisée sous la forme d'une grille : une matrice de dimension $(n \times m)$. Par exemple, la matrice suivante représente une carte de dimensions (5×5) où il n'y a que 3 cases remplies de trésors (les valeurs strictement positives). Les autres cases (valeurs nulles) ne contiennent pas de trésors.

0	0	0	0	10
0	0	0	0	0
0	0	0	0	0
0	0	40	0	0
20	0	0	0	0

3 Votre idée initiale est de stocker cette carte dans une liste de listes, après tout vous avez l'habitude de gérer ainsi des matrices. Encodez en Python la liste correspondant au tableau ci-dessus. Quel problème remarquez-vous ? Pourquoi est-ce ennuyeux ?

4 Vous décidez finalement d'employer une autre solution : un dictionnaire où, pour une paire $(key, value)$, la clé est un tuple (x, y) représentant les coordonnées du trésor et la valeur est la valeur totale des trésors enterrés à ces coordonnées. Encodez en Python le dictionnaire correspondant au tableau ci-dessus. Votre problème de la question précédente est-il résolu ?

Pour aider Edward et pour vous entraîner à manipuler une combinaison nouvelle de structures de données, vous écrivez quelques fonctions pour la manipuler. Pour chacune des fonctions suivantes, vous pouvez considérer que les coordonnées fournies en paramètre sont bel et bien un tuple d'entiers (**précondition**). Vous pensez à :

5 Une fonction $get_treasure(map, coords)$. Cette fonction prend en paramètres les coordonnées *coords* (tuple d'entiers) pour lesquelles il faut récupérer la valeur du trésor. Elle renvoie la valeur contenue dans le dictionnaire *map* s'il y en a une et 0 sinon.

6 Une fonction $add_treasure(map, coords, amount)$. Cette fonction prend en paramètres l'entier *amount* qui est la valeur à ajouter et le tuple d'entiers *coords* qui représente les coordonnées et qui est l'endroit où il faut ajouter la valeur. Cette fonction ajoute la valeur dans notre dictionnaire *map* aux coordonnées indiquées. Si un trésor est déjà présent à cet endroit, il faut ajouter la nouvelle valeur à la précédente. Sinon, il faut créer une nouvelle entrée.

7 Une fonction $set_treasure(map, coords, amount)$. Cette fonction prend en paramètres l'entier *amount* qui est la valeur à affecter et le tuple d'entiers *coords* qui représente les coordonnées et qui est l'endroit où il faut affecter la valeur. Elle affecte la valeur dans notre dictionnaire *map* aux coordonnées indiquées. Si un trésor est déjà présent à cet endroit, il faut remplacer l'ancienne valeur par la nouvelle. Sinon, il faut créer une nouvelle entrée. Dans le cas où la valeur est de 0, il faut que le dictionnaire n'ait pas d'entrée pour ces coordonnées.

Edward souhaite pouvoir voir les informations que donnent deux cartes quand elles sont combinées. Pour ce faire, vous décidez d'additionner les matrices.

8 Implémentez la fonction *add_map*(*map*1, *map*2). Cette fonction prend en paramètres deux dictionnaires, *map*1 et *map*2. Elle additionne les deux matrices représentés par ces deux dictionnaires. La matrice résultante est renvoyée sous la forme d'un nouveau dictionnaire (dans le même format que les précédents). Les deux autres dictionnaires passés en paramètres ne doivent donc absolument pas être modifiées (c'est à dire qu'il ne doit pas y avoir **d'effets de bord**).

1.3 En parlant de maps ...

Une discipline très appréciée de certains programmeurs est le *code golf*. C'est un jeu où les participants peuvent mettre en valeur leurs capacités en essayant de résoudre un problème en utilisant le moins de caractères possibles. Pour vous y initier, la *DS Academy* vous propose une épreuve un peu plus simple ...

9 Pour chacune des tâches suivantes, proposez un code qui effectue la tâche demandée en utilisant le minimum de lignes de code. Si la tâche mentionne une entrée, votre ligne doit pouvoir facilement s'adapter si l'entrée change sans que vous deviez calculer le résultat à la main (par exemple, "$print([(9, 10), (7, 30), (6, 30)])$" n'est pas une réponse correcte à la deuxième tâche ci-dessous).

Notez que pour chaque tâche, une solution en une ligne est possible. En général, celles-ci sont pensées pour utiliser les notions de mapping, filtrage, réduction ou compréhension.

Plutôt que de chercher le résultat en une seule ligne directement, écrivez une première solution et essayez de réduire sa taille progressivement. La quatrième est particulièrement compliquée, si vous prenez plus qu'une ligne pour celle-là, ce n'est pas grave (même si elle est effectivement possible en une ligne).

- Soit une liste de nombres : $[1, 2, 3, 4, 5, 6]$. Affichez une liste qui contient uniquement les carrés des nombres pairs de cette liste. La sortie doit donc être $[4, 16, 36]$ pour cet exemple.
- Soient deux listes de nombres entiers : $[1, 2, 3, 3, 5, 7, 9]$ et $[3, 4, 7, 5, 3, 7, 11]$. Prenez le carré des nombres impairs de ces listes. Trouvez ensuite les carrés qui apparaissent dans les deux. (**Contrainte :** Vous **ne pouvez pas** utiliser une liste autre que celles fournies.) La solution, pour l'exemple donné, serait donc $9, 25, 49$.
- Soient les temps en secondes suivants réalisés par des coureurs : $[620, 450, 800, 390, 720, 550]$. Affichez une liste de tuples sous la forme (*min*, *sec*) qui correspondent à la conversion en minutes et en secondes des temps réalisés ici. Conservez uniquement les temps inférieurs à 10 minutes. Affichez les dans l'ordre décroissant, du temps le plus long au plus rapide. Pour l'exemple donné ci-dessus, l'affichage doit être $[(9, 10), (7, 30), (6, 30)]$.
- Soit une liste de chaîne de caractères à l'intérieur desquelles sont cachés des entiers : ["$abc42de$", "$hello9world$", "$xyz8pq$", "$27abc$", "$100foo$", "$1a2b3c$"]. Vous devez extraire tous les entiers cachés dans ces chaînes de caractère, et conserver uniquement ceux qui sont des multiples de 3. Prenez ensuite le cube de ces nombres. Le résultat final, pour l'exemple donné, serait donc $[74088, 729, 19683, 27]$.

Attention, la *DS Academy* vous rappelle que ces cascades ne sont pas réalisées par des professionnels mais vous conseille quand même de ne pas les reproduire chez vous. C'est à dire, prenez conscience que cet exercice est réalisé dans le cadre d'un apprentissage ludique. Les lignes qui en résultent peuvent être très difficiles à lire et à maintenir. N'utilisez pas ce genre de code dans une réelle application, préférez toujours dans ces cas-là un code lisible !

2 Exercices complémentaires

10 Edward souhaite pouvoir faire une symétrie sur sa carte par rapport à la diagonale principale. Pour ce faire, vous décidez de calculer la matrice transposée.

Implémentez la méthode *transpose_map*(*map*). Elle calcule la matrice transposée de la matrice représentée par le dictionnaire *map* passé en paramètre. La matrice résultante est renvoyée sous la forme d'un nouveau dictionnaire (dans le même format que le précédent). Le dictionnaire passé en paramètre ne doit donc absolument pas être modifié.

11 Edward souhaite pouvoir voir la différence d'informations entre différentes cartes. Pour ce faire, vous décidez de soustraire les matrices.

Implémentez la méthode $substract_map(map1, map2)$. Cette méthode prend en paramètres deux dictionnaires, $map1$ et $map2$. Elle soustrait la matrice représentée par le deuxième dictionnaire à la matrice représentée par le premier dictionnaire. La matrice résultante est renvoyée sous la forme d'un nouveau dictionnaire (dans le même format que les précédents). Les deux autres dictionnaires passés en paramètre ne doivent donc absolument pas être modifiées.